

Práctica 2 Sistema Backend de Float y Alquiler de Vehículos EcoDrive

<https://github.com/Gabi459/ecodrive.git>

reporte.php

```
<?php

// Activamos el modo estricto de tipos
// Tiene que ser lo primero del archivo
declare(strict_types=1);

// Hacemos que se muestren todos los errores mientras programamos
ini_set('display_errors', '1');
ini_set('display_startup_errors', '1');
error_reporting(E_ALL);

// Seguridad en HTML (Bloque 4)
// Esta función protege los datos antes de mostrarlos en la página
// Cambia los símbolos como < > " ' por un código que el navegador
// enseña como texto normal, así nadie puede meter código en la página
function e(string|int $valor): string
{
    return htmlspecialchars((string) $valor, ENT_QUOTES, 'UTF-8');
}

// =====
// BLOQUE 3: Tratamiento de texto multibyte y análisis de flota
// =====

// Catálogo de vehículos con tildes, ñ y otros caracteres especiales
// Algunos tienen descuento o extras, otros los tienen vacíos (null)
// y otros no los tienen, para ver la diferencia entre isset y
array_key_exists
$catalogo = [

    [
        'modelo'    => 'zoé évolution',
        'categoria' => 'utilitario eléctrico',
        'autonomia' => 395,
```

```
'unidades' => 12,
'descuento' => 10,
'extras'    => ['Cargador portátil'],
],

[
  'modelo'    => 'ñandú e-city',
  'categoria' => 'MICRO URBANO',
  'autonomia' => 180,
  'unidades'  => 25,

  // La clave existe, pero su valor es null
  'descuento' => null,
  'extras'    => null,
],

[
  'modelo'    => 'ión trópico suv',
  'categoria' => 'todoterreno compacto',
  'autonomia' => 450,
  'unidades'  => 6,

  // En este vehículo no existen las claves descuento ni extras
],

[
  'modelo'    => 'mégane "côte" d\'azur',
  'categoria' => 'compacto de ciudad',
  'autonomia' => 420,
  'unidades'  => 9,
  'descuento' => 5,
],

[
  // Este nombre contiene HTML a propósito
  // para comprobar que después se muestra de forma segura
  'modelo'    => '<script>alert("XSS")</script> cargo',
  'categoria' => 'furgoneta de reparto',
  'autonomia' => 280,
  'unidades'  => 14,
  'extras'    => null,
],
];
```

```

// -----
// Normalización de textos de flota
// -----

// Usamos las funciones que empiezan por mb_ porque son las que
// funcionan bien con las tildes y la ñ

// Recorremos todos los vehículos del catálogo
foreach ($catalogo as $i => $vehiculo) {

    // Ponemos la primera letra de cada palabra de la categoría en
mayúscula
    // Ejemplo: "MICRO URBANO" -> "Micro Urbano"
    $catalogo[$i]['categoria'] =
mb_convert_case($vehiculo['categoria'], MB_CASE_TITLE, 'UTF-8');

    // Pasamos el nombre del modelo a mayúsculas, también las letras
con tilde
    // Ejemplo: "zoé évolution" -> "ZOÉ ÉVOLUTION"
    $catalogo[$i]['modelo'] = mb_strtoupper($vehiculo['modelo'],
'UTF-8');

    // Contamos cuántas letras tiene el nombre del modelo
    // Con strlen, "zoé" daría 4 porque la é cuenta doble; con
mb_strlen da 3
    $catalogo[$i]['longitud'] = mb_strlen($vehiculo['modelo'],
'UTF-8');
}

// -----
// Ordenación del catálogo
// -----

// Ordenamos los vehículos de mayor a menor autonomía
// El operador <=> compara dos valores y dice si el primero es menor,
igual o mayor
// Ponemos $b antes que $a para que el orden sea de mayor a menor
usort($catalogo, function (array $a, array $b): int {
    return $b['autonomia'] <=> $a['autonomia'];
});

```

```

// -----
// Seguridad en código cliente (Bloque 4)
// -----

// Convertimos el catálogo a JSON para poder usarlo en JavaScript
// Las opciones JSON_HEX cambian los símbolos < > & ' " por códigos
seguros
// Así ningún dato puede romper el código JavaScript ni meter código
malicioso
$jsonFlota = json_encode(
    $catalogo,
    JSON_HEX_TAG | JSON_HEX_AMP | JSON_HEX_APOS | JSON_HEX_QUOT
);

// =====
// BLOQUE 4: Reportes, seguridad multinivel y control de versiones
// =====

// -----
// Captura en memoria
// -----

// A partir de aquí, todo lo que se genere se guarda en memoria
// en lugar de enviarse directamente al navegador
ob_start();

?>
<!DOCTYPE html>
<html lang="es">

<head>

    <!-- Indicamos que usamos UTF-8 para tildes, ñ y otros caracteres
-->
    <meta charset="UTF-8">

    <!-- Título de la página -->
    <title>EcoDrive - Reporte de flota</title>

```

```

</head>

<body>

    <h1>EcoDrive · Reporte de inventario de flota</h1>

    <!-- Tabla de vehículos ordenados por autonomía -->
    <h2>Flota ordenada por autonomía (de mayor a menor)</h2>

    <table border="1">

        <tr>
            <th>Modelo</th>
            <th>Categoría</th>
            <th>Longitud (caracteres)</th>
            <th>Autonomía</th>
            <th>Unidades</th>
        </tr>

        <?php foreach ($catalogo as $v): ?>

            <!-- Mostramos todos los datos con e() para que sea seguro
-->

            <tr>
                <td><?= e($v['modelo']) ?></td>
                <td><?= e($v['categoria']) ?></td>
                <td><?= e($v['longitud']) ?></td>
                <td><?= e($v['autonomia']) ?> km</td>
                <td><?= e($v['unidades']) ?></td>
            </tr>

        <?php endforeach; ?>

    </table>

    <!-- Diferenciación de existencia de propiedades (Bloque 3) -->
    <!-- Comprobamos si cada vehículo tiene descuento y extras -->
    <h2>Atributos opcionales: array_key_exists() frente a isset()</h2>

    <table border="1">

```

```

        <tr>
            <th>Modelo</th>
            <th>Atributo</th>
            <th>array_key_exists()</th>
            <th>isset()</th>
        </tr>

        <?php foreach ($catalogo as $v): ?>
            <?php foreach (['descuento', 'extras'] as $clave): ?>

                <tr>
                    <td><?= e($v['modelo']) ?></td>
                    <td><?= e($clave) ?></td>

                    <!-- array_key_exists solo mira si la clave existe,
aunque esté vacía (null) -->
                    <td><?= array_key_exists($clave, $v) ? 'true' :
'false' ?></td>

                    <!-- isset mira si la clave existe y además tiene
un valor (no es null) -->
                    <!-- Por eso, si descuento es null,
array_key_exists da true e isset da false -->
                    <td><?= isset($v[$clave]) ? 'true' : 'false'
?></td>

                </tr>

            <?php endforeach; ?>
        <?php endforeach; ?>

    </table>

    <!-- Lista que rellenamos con JavaScript -->
    <h2>Datos transferidos a JavaScript</h2>

    <ul id="lista"></ul>

    <script>

        // Recibimos en JavaScript los datos que hemos preparado en PHP
        const flota = <?= $jsonFlota ?>;

```

```

        // Recorremos los vehículos y creamos un li para cada uno
        // Usamos textContent para que se muestre como texto normal y
no se ejecute nada
        flota.forEach((v) => {
            const li = document.createElement('li');
            li.textContent = `${v.modelo} · ${v.autonomia} km`;
            document.getElementById('lista').appendChild(li);
        });

</script>

</body>

</html>
<?php

// Recogemos todo lo que habíamos guardado en memoria
// y lo metemos en una variable con el reporte completo
$reporte = ob_get_clean();

// Ahora sí enviamos el reporte al navegador
echo $reporte;

```

procesador.php

```

<?php
/**
 * EcoDrive - Procesador de alquileres
 *
 * Valida la petición GET, calcula el importe de la reserva,
 * gestiona excepciones y categoriza la tarifa final.
 *
 * Uso: procesador.php?dias=5
 */

// =====
// BLOQUE 1: Configuración del servidor, control de entrada y
diagnóstico
// =====

// Tipado estricto: debe ser la primera sentencia del archivo
declare(strict_types=1);

```

```

// Diagnóstico de errores (solo para desarrollo)
ini_set('display_errors', '1');
ini_set('display_startup_errors', '1');
error_reporting(E_ALL);

header('Content-Type: text/html; charset=UTF-8');

/**
 * Escapa un valor para imprimirlo de forma segura en HTML.
 *
 * @param string|int|float|null $valor Valor a escapar.
 * @return string Valor escapado.
 */
function e(string|int|float|null $valor): string
{
    return htmlspecialchars((string) $valor, ENT_QUOTES |
ENT_SUBSTITUTE | ENT_HTML5, 'UTF-8');
}

// --- Validación de la reserva ---
$diasRecibidos = $_GET['dias'] ?? null;

$dias = filter_var(
    $diasRecibidos,
    FILTER_VALIDATE_INT,
    ['options' => ['min_range' => 1]]
);

if ($dias === false) {
    http_response_code(400);
    $valorMostrado = is_string($diasRecibidos) ? $diasRecibidos :
'(vacío o formato no válido)';
    echo '<h1>400 - Solicitud incorrecta</h1>';
    echo '<p>El parámetro <code>dias</code> debe ser un número entero
positivo. ';
    echo 'Valor recibido: <strong>' . e($valorMostrado) .
'</strong></p>';
    echo '<p>Ejemplo correcto: <code>procesador.php?dias=5</code></p>';
    exit;
}

// =====

```

```
// BLOQUE 2: Procesador de alquileres, excepciones y tarifas
// =====

/**
 * Calcula el importe total de una reserva de vehículos.
 *
 * Recorre las líneas de la reserva y suma, para cada una,
 * precio por día × unidades × días de alquiler.
 *
 * @param array<int, array{modelo: string, precio_dia: float, unidades:
int}> $lineas
 *      Líneas de la reserva (un elemento por modelo de vehículo).
 * @param int $dias Número de días de alquiler (entero positivo).
 *
 * @return float Importe total de la reserva en euros, sin descuentos.
 *
 * @throws InvalidArgumentException Si la reserva no contiene ninguna
línea
 *      o si los días no son positivos.
 */
function calcularTotalReserva(array $lineas, int $dias): float
{
    // Salida temprana: no se puede facturar una reserva vacía
    if (count($lineas) === 0) {
        throw new InvalidArgumentException('La reserva no contiene
ningún vehículo.');
```

```

* Determina la categoría de descuento o suplemento según el importe
total.
*
* @param float $total Importe total de la reserva en euros.
*
* @return array{categoria: string, ajuste: float} Nombre de la
categoría y
*         porcentaje de ajuste (negativo = descuento, positivo =
suplemento).
*/
function categorizarReserva(float $total): array
{
    // Expresión condicional basada en evaluaciones directas
    return match (true) {
        $total >= 5000 => ['categoria' => 'Flota corporativa', 'ajuste'
=> -15.0],
        $total >= 1500 => ['categoria' => 'Cliente preferente',
'ajuste' => -10.0],
        $total >= 500  => ['categoria' => 'Descuento básico',
'ajuste' => -5.0],
        default      => ['categoria' => 'Reserva pequeña (suplemento
de gestión)', 'ajuste' => 8.0],
    };
}

// --- Datos de ejemplo: reservas recibidas ---
$reservas = [
    'Reserva corporativa GreenLogistics' => [
        ['modelo' => 'Zoé Évolution', 'precio_dia' => 39.90,
'unidades' => 10],
        ['modelo' => 'Ión Trópico SUV', 'precio_dia' => 64.50,
'unidades' => 4],
    ],
    'Reserva particular' => [
        ['modelo' => 'Ñandú e-City', 'precio_dia' => 24.00, 'unidades'
=> 1],
    ],
    // Reserva vacía: provoca la excepción de forma intencionada
    'Reserva sin vehículos' => [],
];

// Resultados que se mostrarán después
$resultados = [];

```

```

foreach ($reservas as $nombre => $lineas) {
    try {
        $total = calcularTotalReserva($lineas, $dias);
        $categoria = categorizarReserva($total);
        $totalFinal = $total * (1 + $categoria['ajuste'] / 100);

        $resultados[] = [
            'nombre'      => $nombre,
            'ok'          => true,
            'total'       => $total,
            'categoria'   => $categoria['categoria'],
            'ajuste'      => $categoria['ajuste'],
            'totalFinal'  => $totalFinal,
        ];
    } catch (InvalidArgumentException $ex) {
        $resultados[] = [
            'nombre' => $nombre,
            'ok'     => false,
            'error'  => $ex->getMessage(),
        ];
    }
}
?>
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>EcoDrive - Procesador de alquileres</title>
    <style>
        body { font-family: system-ui, sans-serif; margin: 2rem; color: #1d2b36; }
        table { border-collapse: collapse; margin-bottom: 1.5rem; }
        th, td { border: 1px solid #c7d3dc; padding: .4rem .8rem; text-align: left; }
        th { background: #e8f3ec; }
        pre { background: #f4f6f8; padding: 1rem; border-radius: 6px; overflow-x: auto; }
        .error { color: #b3261e; font-weight: bold; }
    </style>
</head>
<body>
    <h1>EcoDrive · Procesador de alquileres</h1>

```

```

<h2>Trazabilidad de datos (inspección técnica)</h2>
<?php
    // Se captura var_dump y se escapa para no imprimir datos del
usuario sin filtrar
    ob_start();
    var_dump($diasRecibidos, $dias);
    $inspeccion = (string) ob_get_clean();
    ?>
    <pre><?= e($inspeccion) ?></pre>

<h2>Resultado de las reservas (<?= e($dias) ?> días)</h2>
<table>
    <tr>
        <th>Reserva</th>
        <th>Importe base</th>
        <th>Categoría</th>
        <th>Ajuste</th>
        <th>Importe final</th>
    </tr>
    <?php foreach ($resultados as $r): ?>
        <tr>
            <td><?= e($r['nombre']) ?></td>
            <?php if ($r['ok']): ?>
                <td><?= e(number_format($r['total'], 2, ',', '.'))
?> €</td>
                <td><?= e($r['categoria']) ?></td>
                <td><?= e(sprintf('%+.0f %%', $r['ajuste']))
?></td>
                <td><strong><?= e(number_format($r['totalFinal'],
2, ',', '.')) ?> €</strong></td>
            <?php else: ?>
                <td colspan="4" class="error">Excepción capturada:
<?= e($r['error']) ?></td>
            <?php endif; ?>
        </tr>
    <?php endforeach; ?>
</table>

<p><a href="reporte.php">Ver reporte de flota →</a></p>
</body>
</html>

```

README.md

EcoDrive · Sistema backend de flota y alquiler de vehículos

Práctica 2 – Módulo backend en PHP para validar solicitudes de alquiler de vehículos eléctricos, procesar reservas de flotas corporativas y generar reportes de inventario seguros.

Requisitos

- PHP 8.0 o superior (se usan `match` y tipos unión)
- Extensión `mbstring` activada

Estructura

```
...
ecodrive/
├── procesador.php  # Validación GET (HTTP 400), función documentada,
                    excepciones y tarifas
├── reporte.php     # Catálogo multibyte, isset vs array_key_exists,
                    ordenación, búfer y salida segura
└── README.md
...
```

Ejecución

```
```bash
php -S localhost:8000
...

- Reserva válida: http://localhost:8000/procesador.php?dias=5
- Error 400: http://localhost:8000/procesador.php?dias=abc o `?dias=-3`
- Reporte de flota: http://localhost:8000/reporte.php
```

### ## Funcionalidades por bloque

**\*\*Bloque 1 – Configuración y validación.\*\*** `declare(strict\_types=1)`, `display\_errors` y `error\_reporting(E\_ALL)`. El parámetro `dias` se valida con `filter\_var(FILTER\_VALIDATE\_INT)` y `min\_range = 1`; si falla se responde con `http\_response\_code(400)` y `exit`. Los datos capturados se muestran con `var\_dump` dentro de `

```
`.
```

**\*\*Bloque 2 - Facturación y excepciones.\*\*** `calcularTotalReserva()` está documentada con PHPDoc (`@param`, `@return`, `@throws`) y lanza `InvalidArgumentException` si la reserva está vacía. La excepción se captura con `try/catch` en el flujo principal. La categoría de descuento o suplemento se decide con `match (true)`.

**\*\*Bloque 3 - Texto multibyte y flota.\*\***

`mb_convert_case(MB_CASE_TITLE)`, `mb_strtoupper` y `mb_strlen` (comparado con `strlen` para ver la diferencia entre caracteres y bytes). Se compara `array_key_exists()` (la clave existe) con `isset()` (la clave existe y no es `null`). El catálogo se ordena de mayor a menor autonomía con `usort` y el operador `<=>`.

**\*\*Bloque 4 - Reporte y seguridad.\*\*** El reporte se genera dentro de `ob_start()` / `ob_get_clean()` antes de enviarse. Toda salida HTML pasa por `htmlspecialchars(ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8')`, y los datos para JavaScript se codifican con `json_encode` usando `JSON_HEX_TAG | JSON_HEX_AMP | JSON_HEX_APOS | JSON_HEX_QUOT`. El catálogo incluye un nombre con `<script>` a propósito para comprobar que no se ejecuta.